YZM 513 – Yapay Zekâ için Yüksek Performanslı Programlama

Vize Sınavı — Hafta 1–6

Midterm Exam — Weeks 1–6

CEVAP ANAHTARI

Ad Soyad: _____ Öğrenci no: _____ Tarih: _____

Süre 60 dakika, kapalı kitap. Formül sayfası son sayfadadır.
Biçim 25 soru, toplam 100 puan (her soru 4 puan).
Bölüm A Çoktan seçmeli, S1–S13. *Yalnızca bir* harfi daire içine alın.
Bölüm B Boşluk doldurma, S14–S25. Çizgiye kısa bir sözcük, sayı veya ifade yazın.

Time: *60 minutes, closed book; formula sheet on the last page.* ***Format:*** *25 questions, 100 points (4 each).* ***Part A:*** *multiple choice, circle exactly one letter.* ***Part B:*** *fill in the blank, write one short word, number or expression. English text is a help only; the Turkish text is official.*

Bölüm A – Çoktan seçmeli *(Multiple choice)*

[4] 1. Tek çekirdekli CPU hızı, transistör sayısı artmaya devam etmesine rağmen 2005 civarında belirgin biçimde artmayı bıraktı. Ana neden nedir?

Single-core CPU speed stopped improving much around 2005, even though transistor counts kept growing. What was the main reason?

A. Derleyiciler kodu artık optimize edemedi.
B. Ana bellek çok pahalı hâle geldi.
C. Python'a GIL eklendi.
D. Güç tüketimi ve ısınma, saat frekansını sınırladı.

Neden: "Frekans duvarı": saat hızı 3–5 GHz'de yatay seyretti; bu yüzden hız kazancı artık paralellikten gelir.

[4] 2. Bir `cProfile` raporu, `compute_distances` fonksiyonunun çalışma süresinin %90'ını aldığını gösteriyor. Bu fonksiyonun içinde hangi *satırın* yavaş olduğunu hangi araç gösterir?

A `cProfile` *report shows that* `compute_distances` *takes 90% of the run time. Which tool tells you which* line *inside that function is the slow one?*

A. `line_profiler`
B. `timeit`
C. `cProfile` (tekrar çalıştırmak)
D. `multiprocessing.Pool`

Neden: `cProfile` fonksiyon düzeyinde çalışır; `line_profiler` (`kernprof -l -v`) satır düzeyinde çalışır.

[4] 3. NumPy ile softmax hesaplarken `np.exp`'ten önce neden `logits - logits.max(axis=1, keepdims=True)` işlemi yapılır?

Why do we compute `logits - logits.max(axis=1, keepdims=True)` *before* `np.exp` *in a NumPy softmax?*

A. Kodun GPU üzerinde çalışması için.
B. Python döngüsünü vektörleştirilmiş bir işleme çevirmek için.
C. Sonucu değiştirmeden `exp` işleminde taşmayı (overflow) önlemek için.
D. Her satırın toplamını 1 yapmak için.

Neden: `np.exp(1000.)` taşarak `inf` olur ve `nan` üretir. Satır maksimumunu çıkarmak softmax sonucunu değiştirmez.

[4] 4. Numba'daki `@jit` ile `@njit` arasındaki fark nedir?

What is the difference between Numba's `@jit` and `@njit`?

A. İkisi de derlense bile `@njit` her zaman `@jit`'ten hızlıdır.
B. `@njit`, derleyemezse sessizce yavaş "object mode"a düşmek yerine hata verir.
C. `@njit` çok çekirdekli çalışmayı otomatik açar.
D. `@njit` Python dict ve string'lerini derleyebilir; `@jit` derleyemez.

Neden: `@njit`, `@jit(nopython=True)` ile aynıdır. Paralel çalışma için `parallel=True` ve `prange` gerekir.

[4] 5. Hangi durumda yarış durumu (race condition) vardır?

Which situation contains a race condition?

A. İki thread, kilit kullanmadan ortak bir global değişkende `counter += 1` çalıştırır.
B. İki thread aynı NumPy dizisini yalnızca okur.
C. İki thread, kendi ayrı dizilerine yazar.
D. İki thread, aynı `Lock`'u tutarak bir sayacı artırır.

Neden: Ortak durum, en az bir yazma ve senkronizasyon yok. Burada `+=` üç adımdır: yükle, topla, geri yaz.

[4] 6. GPU'yu aynı anda *en fazla 3* thread'in kullanmasını istiyorsunuz. Hangi ilkel (primitive) uygundur?

You want at most 3 *threads to use the GPU at the same time. Which primitive fits?*

A. `threading.Lock()`
B. `threading.Barrier(3)`
C. `threading.Event()`
D. `threading.Semaphore(3)`

Neden: Semafor en fazla N sahibe izin verir; kilit $N = 1$ durumudur.

[4] 7. 8 çekirdekli bir makinede, *saf Python* ile yazılmış bir tokenizer ile milyonlarca belgeyi tokenize etmeniz gerekiyor. Hangi seçenek gerçek hızlanma sağlar?

You must tokenise millions of documents with a tokenizer written in pure Python *on an 8-core machine. Which choice gives real speed-up?*

A. `ThreadPoolExecutor(max_workers=50)`
B. `ProcessPoolExecutor(max_workers=8)`
C. Tüm belgeler üzerinde `asyncio.gather`
D. Her belge için `threading.Thread` ile bir thread

Neden: Saf Python bytecode'u GIL'i tutar; thread ve coroutine'ler birer birer çalışır. Her process'in kendi GIL'i vardır.

[4] 8. Bir LLM API sunucusuna 500 istek göndereceksiniz; her istek çoğunlukla ağı *bekler*. Hangi yaklaşım en iyi ölçeklenir?

You must send 500 prompts to an LLM API server; each request mostly waits *for the network. Which approach scales best?*

A. 500 işçili `ProcessPoolExecutor`
B. 500 işletim sistemi thread'i (yaklaşık 4 GB stack)
C. `await` ve `gather` ile `asyncio`: tek thread, 500 hafif coroutine
D. Her yanıtı bekleyen düz bir `for` döngüsü

Neden: Coroutine yaklaşık KB, thread yaklaşık MB bellek tutar. Çok sayıda eşzamanlı G/Ç beklemesi `asyncio`'nun kullanım alanıdır.

[4] 9. `ProcessPoolExecutor` üzerinde `executor.submit(lambda x: x**2, 10)` çağrısı `PicklingError` ile başarısız olur. Neden?

`executor.submit(lambda x: x**2, 10)` fails with a `PicklingError` on a `ProcessPoolExecutor`. Why?

A. Fonksiyon ve argümanlar worker process'lere pickle ile gönderilir ve lambda'lar pickle'lanamaz.
B. Lambda'lar GIL tarafından çalıştırılamaz.
C. `ProcessPoolExecutor` yalnızca NumPy fonksiyonlarını kabul eder.
D. Havuzda yeterli işçi yoktur.

Neden: Çözüm: `def` ile adlandırılmış bir fonksiyon tanımlamak.

[4] 10. Her biri yaklaşık 1 ms hesaplama yapan 2000 çok küçük görevli bir Dask işi, seri koddan *daha yavaş* çalışıyor. En olası neden nedir?

A Dask job with 2000 very small tasks (about 1 ms of compute each) is slower *than the serial code. What is the most likely cause?*

A. GIL, Dask'in thread kullanmasını engeller.
B. Görev başına ek yük (overhead) hesaplama kadar büyüktür: granülerlik çok ince.
C. Dask, NumPy fonksiyonlarını çalıştıramaz.
D. Ağ çok hızlıdır.

Neden: Çözüm: görevleri kalınlaştırmak (daha büyük chunk); böylece her görev, zamanlanma maliyetinden çok daha uzun hesaplar.

[4] 11. Neden büyük diziyi görevlere doğrudan vermek yerine bir kez `ref = ray.put(big_array)` çağırıp her göreve `ref` veriyoruz?

Why call `ref = ray.put(big_array)` once and pass `ref` to every task, instead of passing `big_array` itself?

A. Dizinin tüm görevler tarafından aynı anda yazılabilir olmasını sağlar.
B. Diziyi GPU'ya taşır.
C. Ray görevlerini actor'a çevirir.
D. Dizi bir kez serileştirilip nesne deposuna (object store) konur; görevler her seferinde kopya yerine bir referans alır.

Neden: İlke: büyük salt-okunur veriyi bir kez bir yere koy, referans ilet. Veri taşımak genellikle asıl maliyettir.

[4] 12. Matris çarpımı bir GPU'dan ReLU'ya göre çok daha fazla fayda sağlar. Nedeni nedir?

A matrix multiplication gains much more from a GPU than a ReLU does. What is the reason?

A. ReLU bir CUDA kernel'i olarak yazılamaz.
B. Tensor Core'lar yalnızca matris çarpımını destekler.
C. Matmul yüksek aritmetik yoğunluğa sahiptir (hesaplama-sınırlı); ReLU yaklaşık 0,125 FLOP/bayta sahiptir (bant genişliği-sınırlı).
D. GPU'nun CPU'dan daha az çekirdeği vardır, bu yüzden basit işlemler yavaştır.

Neden: $n \times n$ float32 matmul için yoğunluk yaklaşık $n/6$ FLOP/bayttır; ReLU taşınan her 8 bayt için bir işlem yapar.

[4] 13. Bir öğrenci `c = cp.dot(a, b)` (4096×4096) çağrısından önce ve sonra `time.perf_counter()` ile süre ölçüyor ve 0,03 ms buluyor. Ne yanlış gitti?

A student times `c = cp.dot(a, b)` (4096×4096) with `time.perf_counter()` before and after the call, and gets 0.03 ms. What went wrong?

A. `synchronize()` yoktu; bu yüzden yalnızca eşzamansız kernel'in *başlatılması* ölçüldü.
B. Matrisler float64 idi.
C. CuPy `dot`'u desteklemez.
D. Diziler hâlâ CPU belleğindeydi.

Neden: Saati okumadan önce senkronize edin (veya CUDA event kullanın). Aksi hâlde her işlem yaklaşık 20 μs sürüyor gibi görünür.

Bölüm B – Boşluk doldurma *(Fill in the blank)*

[4] 14. İki $n \times n$ matrisin çarpımı yaklaşık _____ $2n^3$ _____ kayan noktalı işlem (FLOP) gerektirir.

Multiplying two $n \times n$ matrices costs about _____ floating-point operations.

Neden: n^2 çıktının her biri için n çarpma ve n toplama gerekir: $2n^3$.

[4] 15. Bir programın seri oranı %20'dir. Amdahl yasasına göre, işlemci sayısı sınırsız olsa bile hızlanma en fazla _____ **5** _____× olabilir.

A program has a serial fraction of 20%. By Amdahl's law, even with an unlimited number of processors the speed-up can never exceed _____×.

Neden: $1/f_s = 1/0{,}2 = 5$.

[4] 16. NumPy'da `b = a[10:20]` bir kopya değil, bir __ **view (görünüm)** __ oluşturur; bu yüzden `b[0] = 999` `a[10]`'u da değiştirir.

In NumPy, `b = a[10:20]` creates a _____, not a copy, so `b[0] = 999` also changes `a[10]`.

Neden: Dilimler ve `.T` belleği paylaşır. Bağımsız dizi için `.copy()` kullanılır.

[4] 17. 1024×1024 boyutlu bir `float32` dizisi **4** MB bellek kaplar (aynı boyutlu `float64` dizisi 8 MB kaplar).

A `float32` array of shape 1024×1024 takes _____ MB of memory (a `float64` array of the same shape takes 8 MB).

Neden: 2^{20} eleman $\times$ 4 bayt = 4 MB.

[4] 18. Yarış durumu için üç şey gerekir: ortak durum (shared state), senkronizasyon olmaması ve en az bir **yazma** erişimi.

A race condition needs three things: shared state, no synchronisation, and at least one _____ access.

Neden: Birçok thread'in aynı veriyi yalnızca okuması güvenlidir.

[4] 19. İki kilit arasındaki deadlock, her thread kilitleri aynı **sırayla** alırsa önlenebilir.

Deadlock between two locks can be avoided if every thread acquires the locks in the same _____.

Neden: Kilit sıralaması: Thread 1'in A'yı tutup B'yi, Thread 2'nin B'yi tutup A'yı beklemesi gerçekleşemez.

[4] 20. GIL vardır çünkü CPython belleği **referans** sayımı ile yönetir; iki thread sayaçları aynı anda güncellerse sayaçlar bozulurdu.

The GIL exists because CPython manages memory with _____ counting, which would be corrupted if two threads updated the counts at the same time.

Neden: Tek büyük kilit, her nesnenin referans sayacı dâhil tüm yorumlayıcı durumunu korur.

[4] 21. Seri oran $f_s = 0{,}05$ ve iletişim oranı $f_c = 0{,}05$ ise, kaç işçi eklenirse eklensin hızlanma en fazla **10** $\times$ olabilir.

With a serial fraction $f_s = 0.05$ and a communication fraction $f_c = 0.05$, the speed-up can never exceed _____$\times$, however many workers are added.

Neden: Üst sınır $= 1/(f_s + f_c) = 1/0{,}10 = 10$.

[4] 22. Ray'de, çağrılar arasında durumunu koruyan bir sınıfa (örneğin bir kez yüklenip birçok tahminde kullanılan bir model) **actor** denir.

In Ray, a class whose instance keeps state between calls (for example a model loaded once and used for many predictions) is called an _____.

Neden: Durumsuz fonksiyonlar *task*'tir; durumlu işçiler *actor*'dır (bir sınıfa `@ray.remote` uygulanır). "Aktör" yanıtı da kabul edilir.

[4] 23. GPU, thread'lerini ____**32**____'şerli gruplar hâlinde çalıştırır; bu gruba *warp* denir. Bir warp'taki thread'ler farklı dallara giderse iki yol da sırayla çalıştırılır ("divergence").

A GPU runs its threads in groups of _____ called a warp*. If threads of a warp take different branches, both paths are executed one after the other ("divergence").*

Neden: Warp, tek bir komutu eş zamanlı yürüten 32 thread'dir.

[4] 24. Bir GPU eğitim döngüsünde her yinelemede `float(loss)` çağırmak, GPU→CPU yönünde zorunlu bir __**senkronizasyon**__ oluşturur ve Python beklerken GPU boşta kalır.

In a GPU training loop, calling `float(loss)` *in every iteration forces a GPU→CPU _____ and the GPU sits idle while Python waits.*

Neden: "Sync" veya "senkronizasyon" kabul edilir. Çözüm: kaybı cihaz üzerinde biriktirip epoch başına bir kez okumak.

[4] 25. 4096×4096 boyutlu bir `float32` matrisi yaklaşık ____**64**____ MB GPU belleği kullanır.

A 4096×4096 `float32` *matrix uses about _____ MB of GPU memory.*

Neden: $4096^2 \times 4$ bayt = 64 MB. PCIe üzerinden ∼12 GB/s ile aktarımı yaklaşık 5 ms sürer.

Formül sayfası *(Formula sheet)*

Amdahl yasası (f_s = seri oran, N = işçi sayısı):

$$S(N) = \frac{1}{f_s + \dfrac{1 - f_s}{N}} \qquad S(\infty) = \frac{1}{f_s}$$

İletişimli Amdahl (f_c = ek yük ve veri taşımaya harcanan oran):

$$S_{comm}(N) = \frac{1}{f_s + f_c + \dfrac{1 - f_s - f_c}{N}} \qquad S_{comm}(\infty) = \frac{1}{f_s + f_c}$$

Matris çarpımı ($n \times n$ iki matris):

$$\text{FLOP} = 2n^3 \qquad \text{GFLOP/s} = \frac{2n^3}{t \cdot 10^9}$$

Bellek ve aktarım:

$$\text{bayt} = \text{eleman sayısı} \times \text{eleman başına bayt} \quad (\texttt{float32} = 4,\ \texttt{float64} = 8)$$

$$t_{\text{aktarım}} = \frac{\text{bayt}}{\text{bant genişliği}}$$

Aritmetik yoğunluk $= \dfrac{\text{FLOP}}{\text{taşınan bayt}}$; $n \times n$ float32 matmul için yaklaşık $n/6$.

Birimler: bu sınavda 1 MB $= 2^{20}$ bayt (1024×1024). Tipik PCIe 3.0 bant genişliği: $\sim$12 GB/s.